

Министерство науки и высшего образования
Российской Федерации
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»
ИРИТ-РТФ
Школа магистратуры

Лабораторная работа №7

«Система ввода/вывода в Java. Работа с файлами через байтовые потоки»

Преподаватель

Архипов Н.А.

Студент

Жунёв А.А.

Группа РИМ-150950

Екатеринбург 2026

Цель работы

Получить навыки работы с каталогами и файлами операционной системы, а также с классами ввода/вывода Java. Закрепить навыки ввода и вывода данных через байтовые и символьные потоки, использования буферизации, преобразования потоков, сериализации объектов и решения алгоритмических задач.

Ссылка на репозиторий: https://github.com/2made1ra/JAVA_SBER_2026

Код лабораторной работы расположен в пакете: `java-core-2026/src/lab7`

1. Описание задания

В рамках лабораторной работы необходимо выполнить следующие пункты:

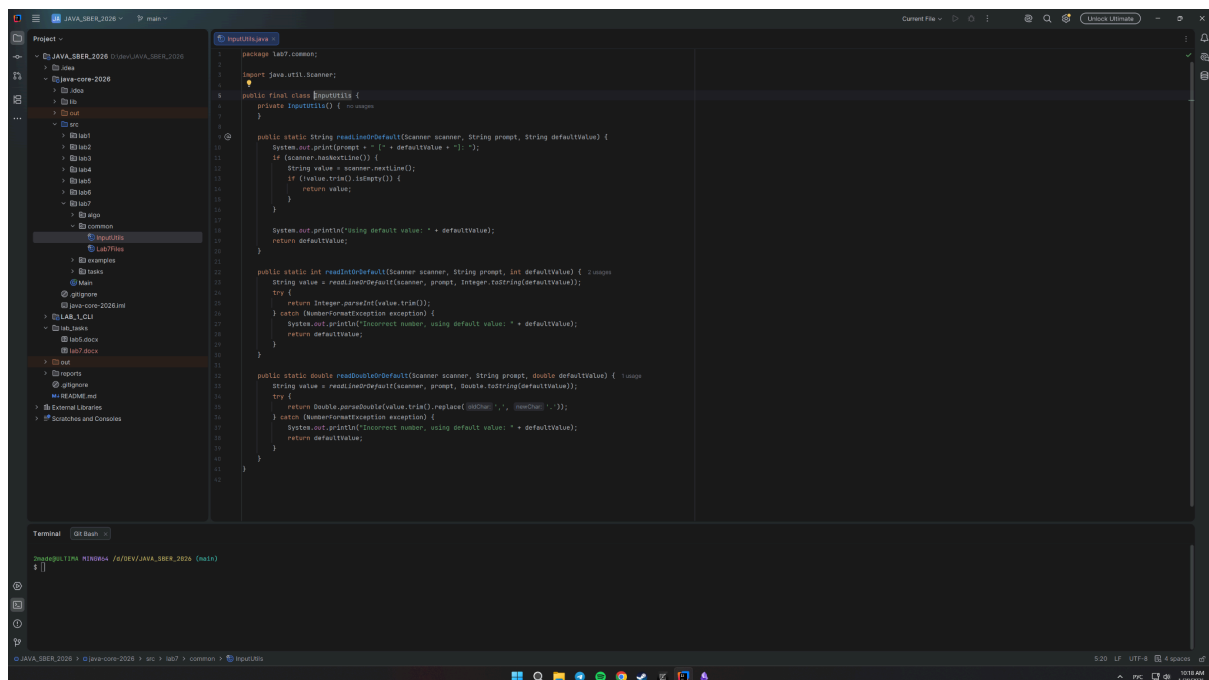
1. Воспроизвести примеры программ из раздела 1 «Примеры программ, работающих с файлами».
2. Доработать примеры так, чтобы данные, записываемые в файл, можно было вводить с консоли.
3. Создать программу, которая считывает текстовый файл и выводит количество строк.
4. Написать программу для копирования содержимого одного текстового файла в другой файл.
5. Создать приложение, которое запрашивает название файла и выводит его размер в байтах.
6. Написать программу, которая запрашивает имя файла и слово для поиска, а затем выводит все строки, содержащие это слово.
7. Создать приложение, которое запрашивает название файла и текст для записи, после чего выводит количество записанных символов.
8. Реализовать сохранение объекта класса в файл и восстановление объекта из файла через сериализацию.
9. Решить две алгоритмические задачи. Для выполнения этого пункта были выбраны задачи из CodeRun по предоставленным скриншотам:
 - «Средний элемент»;
 - «Ход конём».

2. Ход выполнения

2.1. Общая обвязка лабораторной работы

Для повторяющихся операций были добавлены вспомогательные классы:

- `java-core-2026/src/lab7/common/InputUtils.java` — чтение строк, целых чисел и вещественных чисел из консоли с возможностью использовать значение по умолчанию.
- `java-core-2026/src/lab7/common/Lab7Files.java` — создание временных каталогов и файлов для демонстрационных запусков лабораторной работы.



The screenshot shows an IDE with a project named 'JAVA_SBER_2026'. The file explorer on the left shows the project structure, including 'src' and 'common' directories. The main editor displays the code for 'InputUtils.java'. The code defines a package 'lab7.common' and imports 'java.util.Scanner'. It contains three static methods: 'readStringDefault', 'readIntDefault', and 'readDoubleDefault'. Each method uses a 'Scanner' to read input from the console, with a default value provided. The 'readStringDefault' method uses 'nextLine()' and 'trim()' to get a string. The 'readIntDefault' method uses 'nextInt()' and 'Integer.toString()' to get an integer. The 'readDoubleDefault' method uses 'nextDouble()' and 'Double.toString()' to get a double. All methods have a 'try-catch' block to handle 'NoSuchElementException' and 'NumberFormatException'. The terminal at the bottom shows the command 'mvn compile' being executed, resulting in a successful build.

```
package lab7.common;

import java.util.Scanner;

public final class InputUtils {

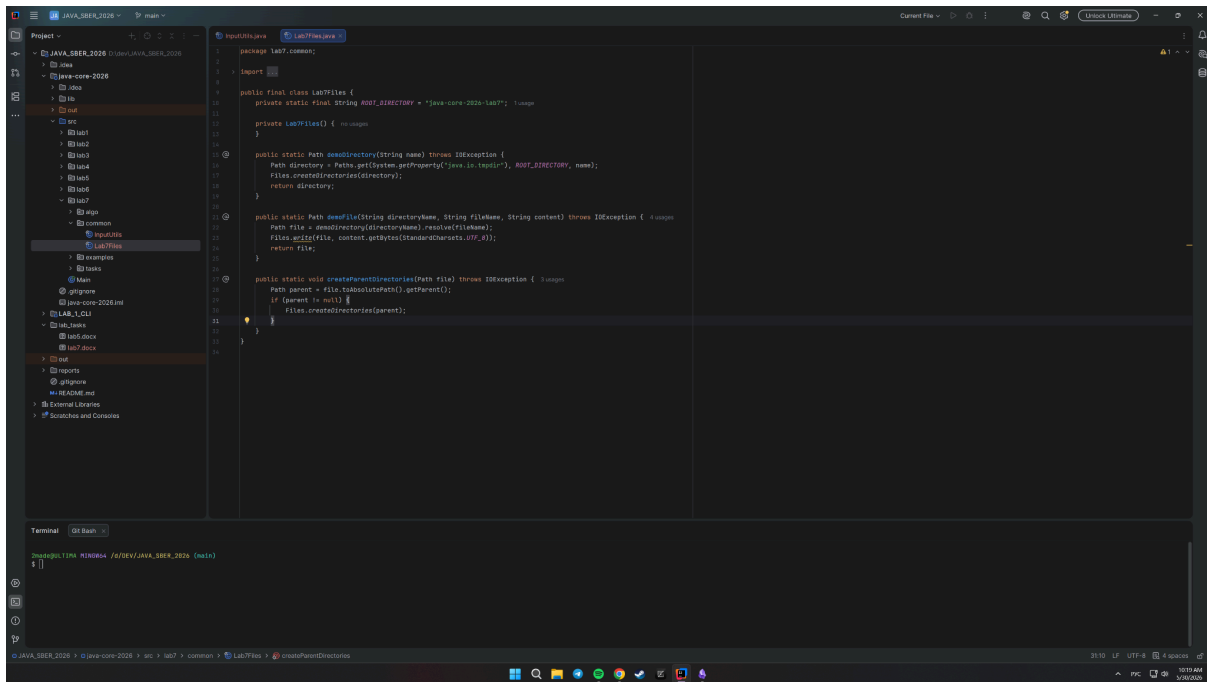
    private InputUtils() { }

    public static String readStringDefault(Scanner scanner, String prompt, String defaultValue) {
        System.out.print(prompt + " [" + defaultValue + "]: ");
        if (scanner.hasNextLine()) {
            String value = scanner.nextLine();
            if (value.trim().isEmpty()) {
                return defaultValue;
            }
            return value;
        }
        System.out.println("using default value: " + defaultValue);
        return defaultValue;
    }

    public static int readIntDefault(Scanner scanner, String prompt, int defaultValue) {
        String value = readStringDefault(scanner, prompt, Integer.toString(defaultValue));
        try {
            return Integer.parseInt(value.trim());
        } catch (NumberFormatException exception) {
            System.out.println("Incorrect number, using default value: " + defaultValue);
            return defaultValue;
        }
    }

    public static double readDoubleDefault(Scanner scanner, String prompt, double defaultValue) {
        String value = readStringDefault(scanner, prompt, Double.toString(defaultValue));
        try {
            return Double.parseDouble(value.trim().replaceAll("\\s+", ""));
        } catch (NumberFormatException exception) {
            System.out.println("Incorrect number, using default value: " + defaultValue);
            return defaultValue;
        }
    }
}
```

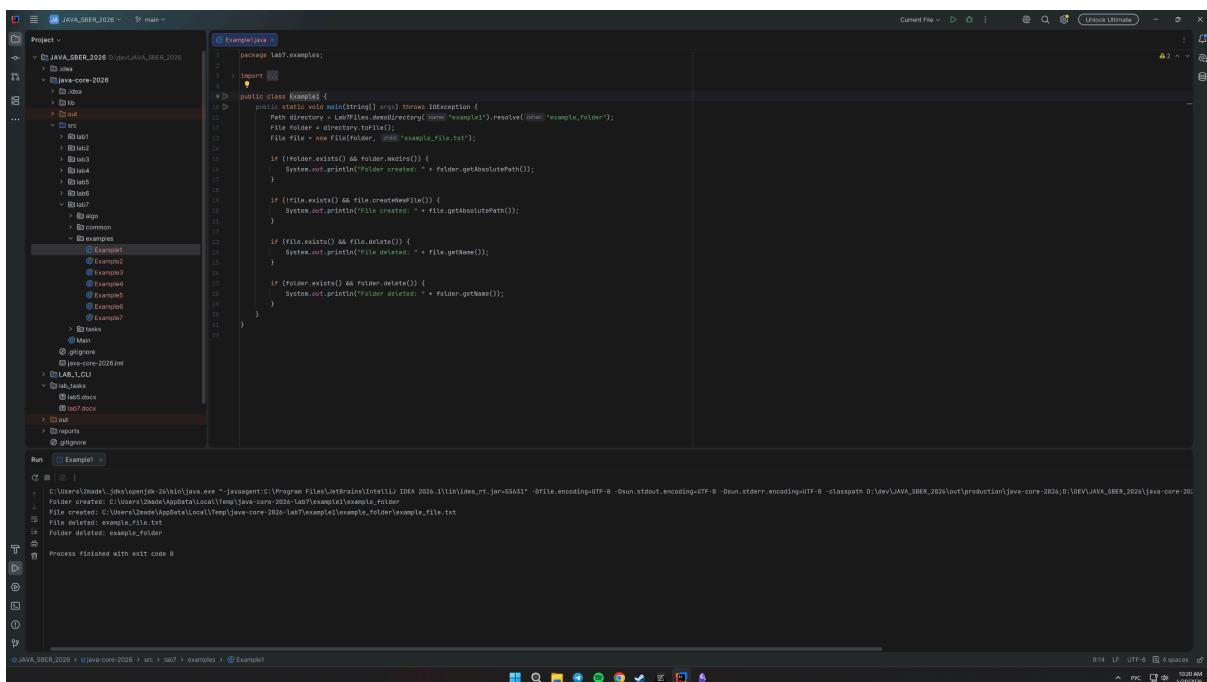
Terminal: `mvn compile`



2.2. Пример 1. Работа с классом File

Файл с решением: [java-core-2026/src/lab7/examples/Example1.java](#)

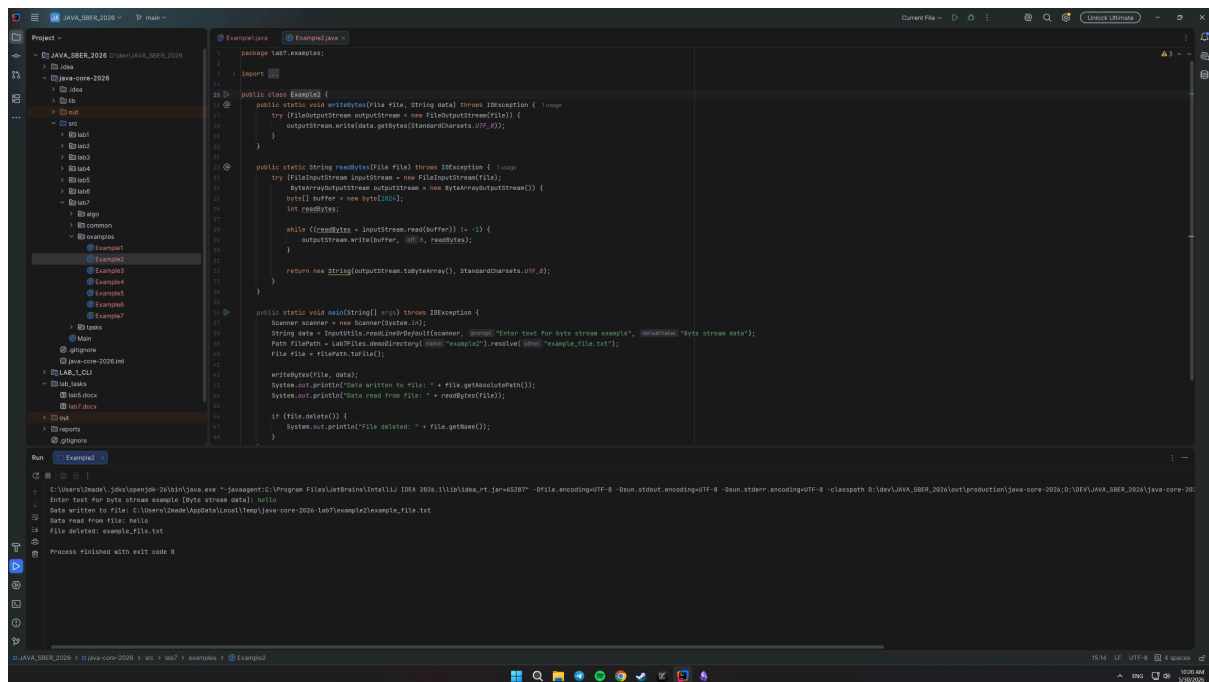
В программе создается каталог **example_folder**, внутри него создается файл **example_file.txt**. После проверки существования файл и каталог удаляются. В консоль выводится ход выполнения операций.



2.3. Пример 2. Байтовый ввод и вывод данных

Файл с решением: [java-core-2026/src/lab7/examples/Example2.java](#)

Пример использует **FileOutputStream** для записи строки в файл и **FileInputStream** для чтения данных из файла. Данные для записи можно ввести с консоли, при пустом вводе используется значение по умолчанию.



```
1 package lab7.examples;
2
3 import java.io.*;
4
5 public class Example2 {
6
7     public static void writeBytes(File file, String data) throws IOException {
8         try (FileOutputStream outputStream = new FileOutputStream(file)) {
9             outputStream.write(data.getBytes(StandardCharsets.UTF_8));
10        }
11    }
12
13     public static String readBytes(File file) throws IOException {
14         try (FileInputStream inputStream = new FileInputStream(file);
15              ByteArrayOutputStream outputStream = new ByteArrayOutputStream()) {
16             byte[] buffer = new byte[1024];
17             int bytesRead;
18             while ((bytesRead = inputStream.read(buffer)) != -1) {
19                 outputStream.write(buffer, 0, bytesRead);
20             }
21             return new String(outputStream.toByteArray(), StandardCharsets.UTF_8);
22         }
23     }
24
25     public static void main(String[] args) throws IOException {
26         Scanner scanner = new Scanner(System.in);
27         String data = "Enter text for byte stream example";
28         Path filePath = Paths.get("examples/example_file.txt");
29         File file = filePath.toFile();
30
31         writeBytes(file, data);
32         System.out.println("Data written to file: " + file.getAbsolutePath());
33         System.out.println("Data read from file: " + readBytes(file));
34
35         if (file.delete()) {
36             System.out.println("File deleted: " + file.getName());
37         }
38     }
39 }
```

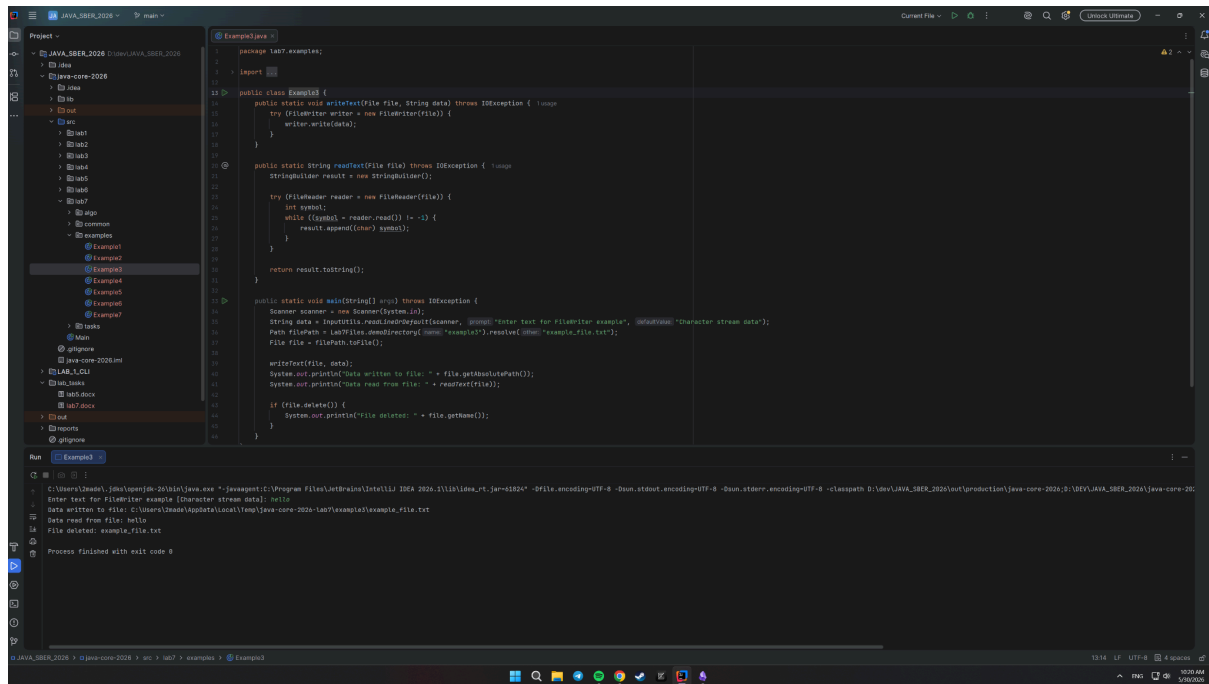
Run

```
Process finished with exit code 0
Enter text for byte stream example [byte stream data]: hello
Data written to file: C:\Users\Idea\Idea\src\main\java-core-2026-lab7\examples\example_file.txt
Data read from file: hello
File deleted: example_file.txt
```

2.4. Пример 3. Символьные потоки FileReader и FileWriter

Файл с решением: [java-core-2026/src/lab7/examples/Example3.java](#)

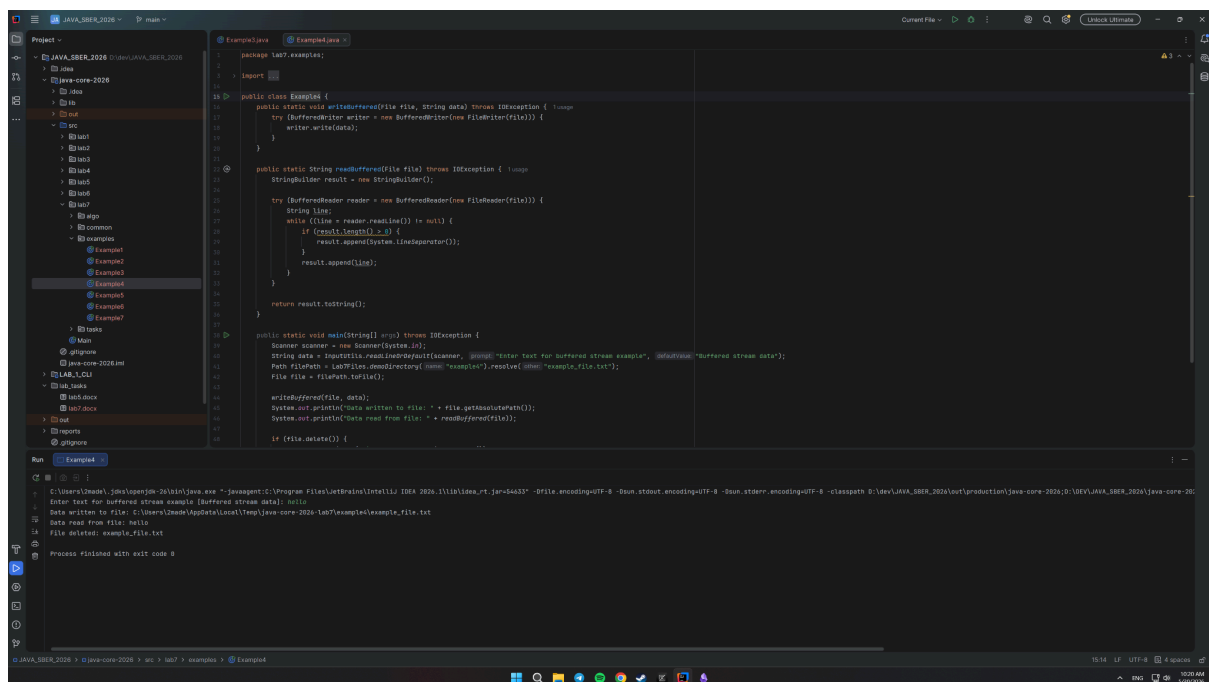
Программа записывает строку в файл через **FileWriter**, затем читает содержимое через **FileReader**. Для чтения используется посимвольный проход по файлу.



2.5. Пример 4. Буферизация ввода и вывода

Файл с решением: [java-core-2026/src/lab7/examples/Example4.java](#)

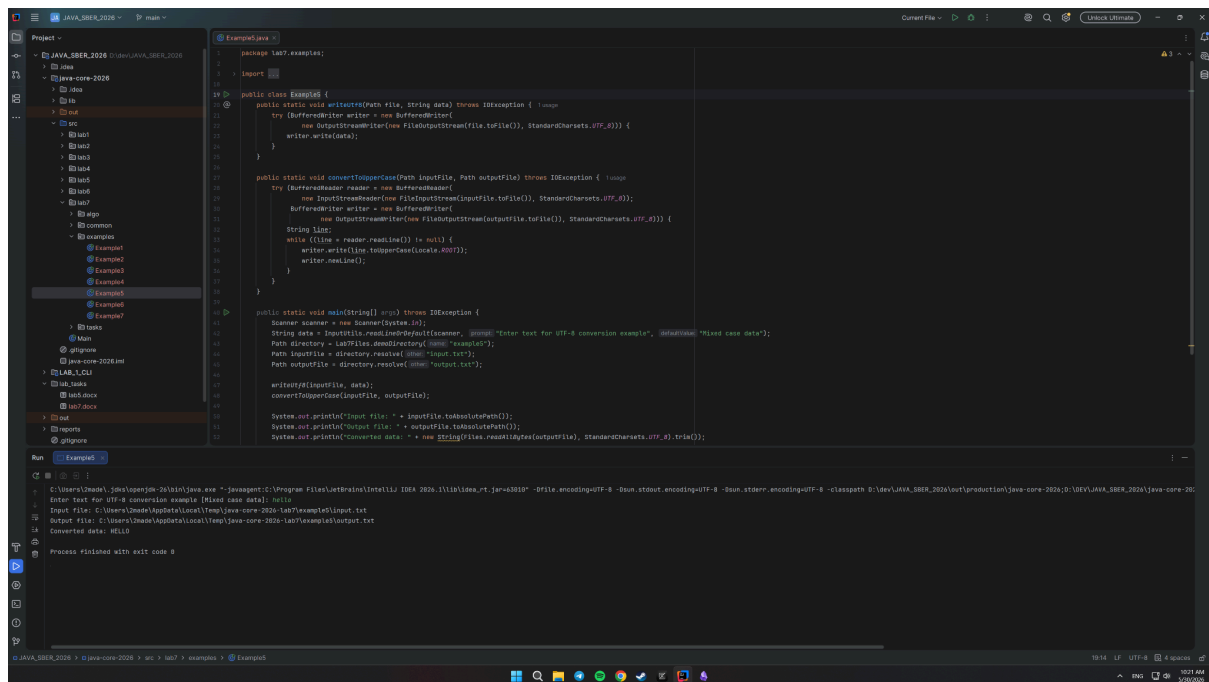
В примере используются **BufferedWriter** и **BufferedReader**, оборачивающие символьные потоки. Буферизация уменьшает количество прямых обращений к файловой системе и делает чтение/запись удобнее.



2.6. Пример 5. Преобразование байтовых потоков в символные

Файл с решением: [java-core-2026/src/lab7/examples/Example5.java](#)

Программа записывает текст в UTF-8, читает его через `InputStreamReader`, преобразует строки в верхний регистр и записывает результат через `OutputStreamWriter`.



```
package lab7.examples;

import java.io.*;

public class Example5 {

    public static void writeToUTF8(Path file, String data) throws IOException {
        try (BufferedWriter writer = new BufferedWriter(
            new OutputStreamWriter(new FileOutputStream(file.toFile()), StandardCharsets.UTF_8))) {
            writer.write(data);
        }
    }

    public static void convertToUpperCase(Path inputFile, Path outputFile) throws IOException {
        try (BufferedReader reader = new BufferedReader(
            new InputStreamReader(new FileInputStream(inputFile.toFile()), StandardCharsets.UTF_8));
            BufferedWriter writer = new BufferedWriter(
            new OutputStreamWriter(new FileOutputStream(outputFile.toFile()), StandardCharsets.UTF_8))) {
            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line.toUpperCase(Locale.ROOT));
                writer.newLine();
            }
        }
    }

    public static void main(String[] args) throws IOException {
        Scanner scanner = new Scanner(System.in);
        String data = scanner.nextLine();
        Path directory = Paths.get("examples");
        Path inputFile = directory.resolve("input.txt");
        Path outputFile = directory.resolve("output.txt");

        writeToUTF8(inputFile, data);
        convertToUpperCase(inputFile, outputFile);

        System.out.println("Input file: " + inputFile.toAbsolutePath());
        System.out.println("Output file: " + outputFile.toAbsolutePath());
        System.out.println("Converted data: " + new String("File reading bytes from output file".getBytes(StandardCharsets.UTF_8), StandardCharsets.UTF_8));
    }
}
```

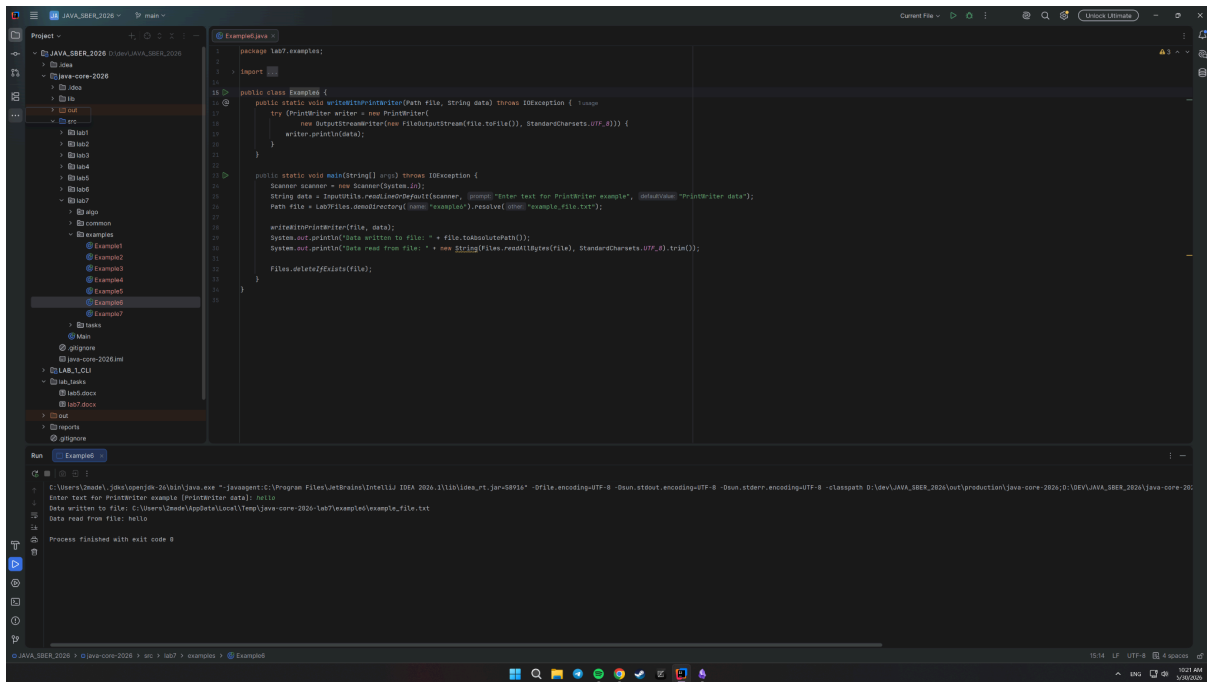
Run

```
C:\Users\user> java -cp .\src\lab7\examples\Example5.jar Example5
Enter text for UTF-8 conversion example (Please use data): hello
Input file: C:\Users\user\AppData\Local\Temp\java-core-2026-lab7\examples\input.txt
Output file: C:\Users\user\AppData\Local\Temp\java-core-2026-lab7\examples\output.txt
Converted data: HELLO
Process finished with exit code 0
```

2.7. Пример 6. Использование PrintWriter

Файл с решением: [java-core-2026/src/lab7/examples/Example6.java](#)

Пример демонстрирует запись строки в файл через `PrintWriter`. Для задания кодировки используется `OutputStreamWriter` с `StandardCharsets.UTF_8`.



```
1 package lab7.examples;
2
3 import java.io.*;
4
5 public class Example7 {
6
7     public static void writeToFile(Path file, String data) throws IOException {
8         try {
9             FileWriter writer = new FileWriter(file);
10             new BufferedWriter(writer).write(data);
11             writer.close();
12         }
13     }
14
15     public static void main(String[] args) throws IOException {
16         Scanner scanner = new Scanner(System.in);
17         String data = InputUtils.readLine(scanner, "Enter text for PrintWriter example", "Default");
18         Path file = Lab7Utils.resolve(scanner, "example", "example_file.txt");
19
20         writeToFile(file, data);
21         System.out.println("Data written to file: " + file.toAbsolutePath());
22         System.out.println("Data read from file: " + new String(Files.readAllBytes(file), StandardCharsets.UTF_8).trim());
23         Files.deleteIfExists(file);
24     }
25 }
```

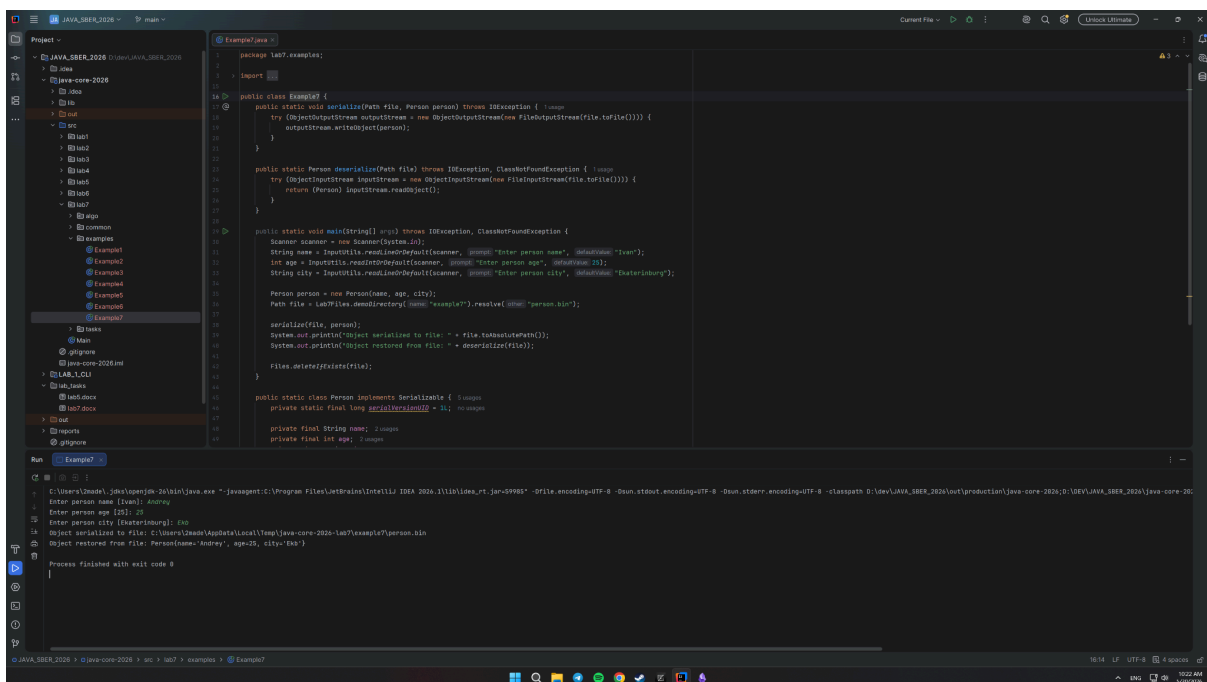
Run

```
C:\Users\Izabela\AppData\Local\Temp\jrun-2026-11\lib\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.1\lib\idea_rt.jar=50951 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath D:\dev\JAVA_SBER_2026\out\production\java-core-2026\0\dev\JAVA_SBER_2026\java-core-2026.jar
Enter text for PrintWriter example (PrintWriter data): hello
Data written to file: C:\Users\Izabela\AppData\Local\Temp\java-core-2026-lab7\example\example_file.txt
Data read from file: hello
Process finished with exit code 0
```

2.8. Пример 7. Сериализация объекта

Файл с решением: [java-core-2026/src/lab7/examples/Example7.java](#)

В примере создан вложенный класс **Person**, реализующий интерфейс **Serializable**. Объект записывается в бинарный файл через **ObjectOutputStream**, затем восстанавливается через **ObjectInputStream**.



```
1 package lab7.examples;
2
3 import java.io.*;
4
5 public class Example7 {
6
7     public static void serialize(Path file, Person person) throws IOException {
8         try {
9             ObjectOutputStream outputStream = new ObjectOutputStream(new FileOutputStream(file.toFile()));
10             outputStream.writeObject(person);
11         }
12     }
13
14     public static Person deserialize(Path file) throws IOException, ClassNotFoundException {
15         try {
16             ObjectInputStream inputStream = new ObjectInputStream(new FileInputStream(file.toFile()));
17             return (Person) inputStream.readObject();
18         }
19     }
20
21     public static void main(String[] args) throws IOException, ClassNotFoundException {
22         Scanner scanner = new Scanner(System.in);
23         String name = InputUtils.readLine(scanner, "Enter person name", "DefaultName");
24         int age = InputUtils.readInt(scanner, "Enter person age", "DefaultAge");
25         String city = InputUtils.readLine(scanner, "Enter person city", "DefaultCity");
26
27         Person person = new Person(name, age, city);
28         Path file = Lab7Utils.resolve(scanner, "example", "person.bin");
29
30         serialize(file, person);
31         System.out.println("Object serialized to file: " + file.toAbsolutePath());
32         System.out.println("Object restored from file: " + deserialize(file));
33         Files.deleteIfExists(file);
34     }
35
36     public static class Person implements Serializable {
37         private static final long serialVersionUID = 1L;
38         private final String name;
39         private final int age;
40         private final String city;
41     }
42 }
```

Run

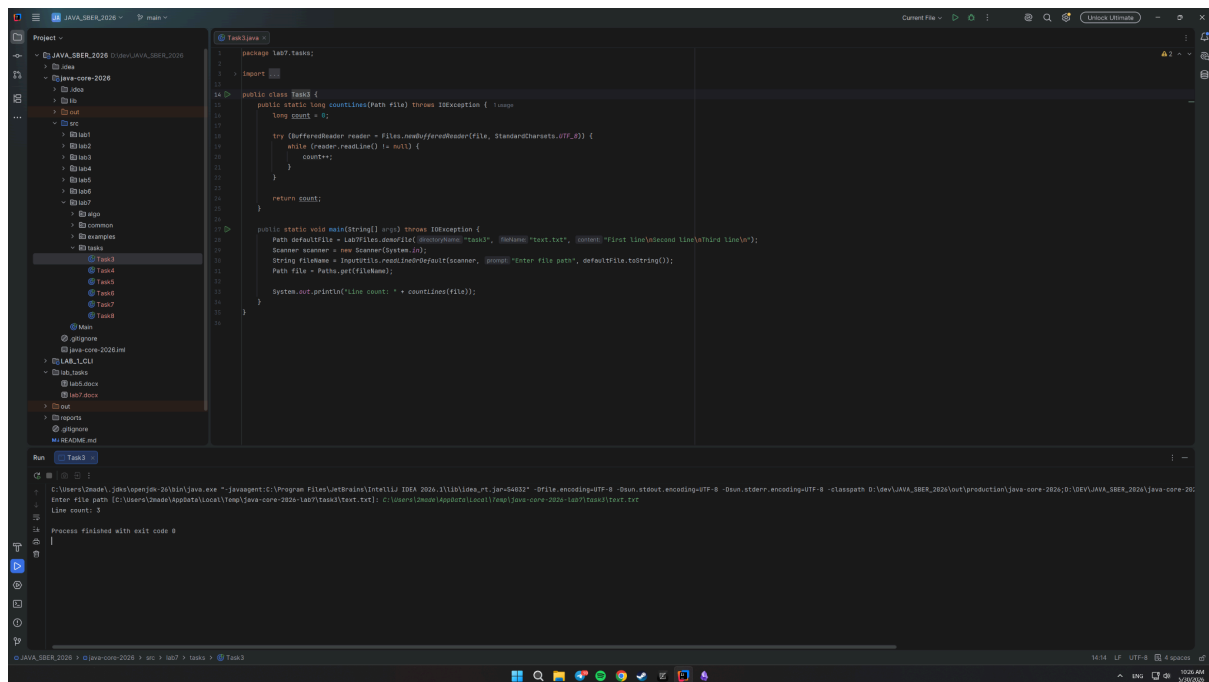
```
C:\Users\Izabela\AppData\Local\Temp\jrun-2026-11\lib\java.exe -javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2024.1\lib\idea_rt.jar=50955 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath D:\dev\JAVA_SBER_2026\out\production\java-core-2026\0\dev\JAVA_SBER_2026\java-core-2026.jar
Enter person name [name]: Andrey
Enter person age [age]: 23
Enter person city [city]: Katerinburg
Object serialized to file: C:\Users\Izabela\AppData\Local\Temp\java-core-2026-lab7\example\person.bin
Object restored from file: Person{name='Andrey', age=23, city='Katerinburg'}
Process finished with exit code 0
```

3. Задания для самостоятельного выполнения

3.1 Задание 3. Подсчет количества строк в файле

Файл с решением: [java-core-2026/src/lab7/tasks/Task3.java](#)

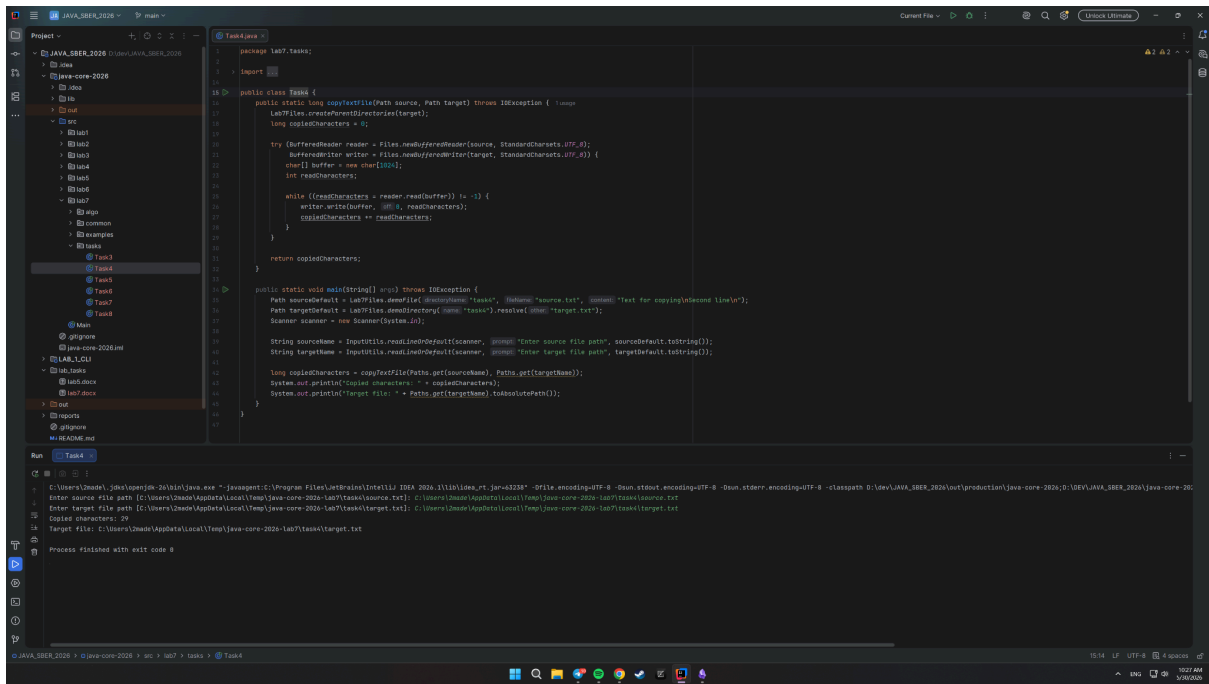
Реализован метод `countLines`, который открывает файл через `Files.newBufferedReader` и считает количество строк, пока `readLine()` не вернет `null`.



3.2 Задание 4. Копирование текстового файла

Файл с решением: [java-core-2026/src/lab7/tasks/Task4.java](#)

Реализован метод `copyTextFile`, который читает исходный файл в буфер символов и записывает данные в целевой файл. Метод возвращает количество скопированных символов.



```
1 package lab7.tasks;
2 import java.io.*;
3
4 public class Task4 {
5     public static long copyFile(Path source, Path target) throws IOException {
6         long copiedCharacters = 0;
7
8         try (BufferedReader reader = Files.newBufferedReader(source, StandardCharsets.UTF_8);
9              BufferedWriter writer = Files.newBufferedWriter(target, StandardCharsets.UTF_8)) {
10             char[] buffer = new char[1024];
11             int readCharacters;
12
13             while ((readCharacters = reader.read(buffer)) != -1) {
14                 writer.write(buffer, 0, readCharacters);
15                 copiedCharacters += readCharacters;
16             }
17
18             return copiedCharacters;
19         }
20     }
21
22     public static void main(String[] args) throws IOException {
23         Path sourceDefault = Lab7Files.getDefaultPath("tasks", "source.txt", "source.txt");
24         Path targetDefault = Lab7Files.getDefaultPath("tasks", "target.txt", "target.txt");
25         Scanner scanner = new Scanner(System.in);
26
27         String sourceName = InputUtils.readLineDefault(scanner, "Enter source file path", sourceDefault.toString());
28         String targetName = InputUtils.readLineDefault(scanner, "Enter target file path", targetDefault.toString());
29
30         long copiedCharacters = copyFile(Path.of(sourceName), Path.of(targetName));
31         System.out.println("Copied characters: " + copiedCharacters);
32         System.out.println("Target file: " + Path.of(targetName).toAbsolutePath());
33     }
34 }
```

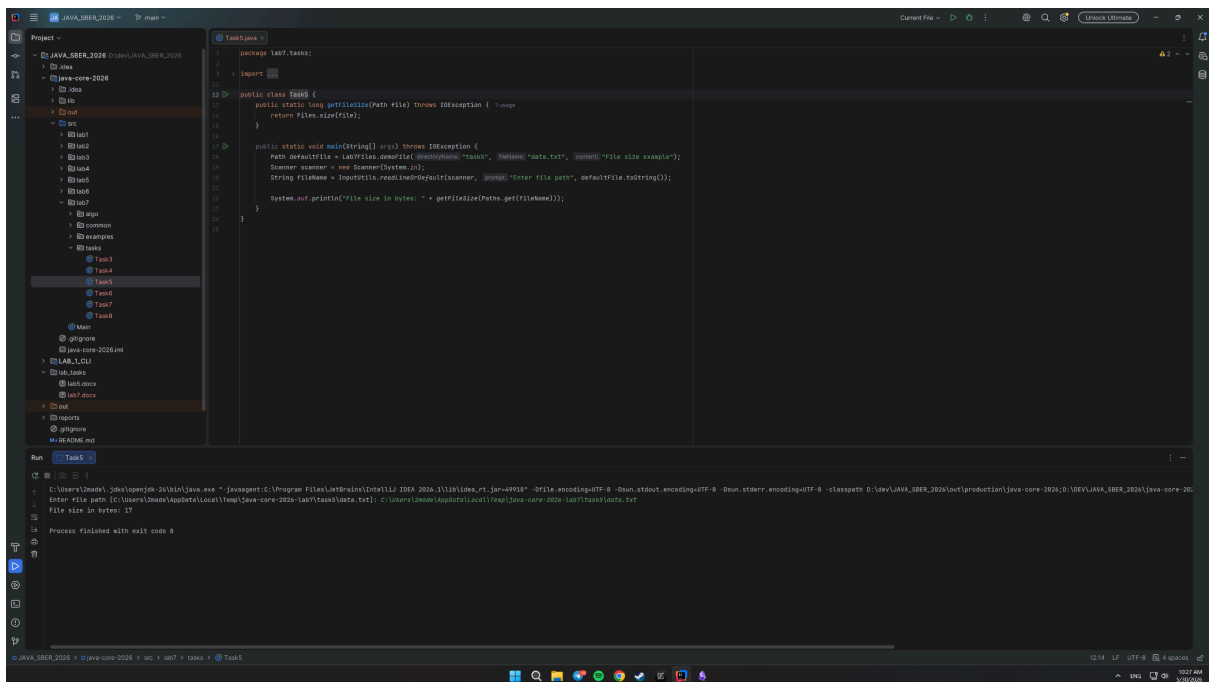
Run Task4

```
C:\Users\Izabela> javac -d bin\src Task4.java
C:\Users\Izabela> java -cp bin\src Task4
Enter source file path C:\Users\Izabela\AppData\Local\Temp\java-core-2026-lab7\task4\source.txt
Enter target file path C:\Users\Izabela\AppData\Local\Temp\java-core-2026-lab7\task4\target.txt
Copied characters: 29
Target file: C:\Users\Izabela\AppData\Local\Temp\java-core-2026-lab7\task4\target.txt
Process finished with exit code 0
```

3.3 Задание 5. Получение размера файла

Файл с решением: [java-core-2026/src/lab7/tasks/Task5.java](#)

Размер файла определяется методом **Files.size**. Путь к файлу вводится пользователем с консоли, при пустом вводе создается демонстрационный файл.



```
1 package lab7.tasks;
2 import java.io.*;
3
4 public class Task5 {
5     public static long getFileSize(Path file) throws IOException {
6         return Files.size(file);
7     }
8
9     public static void main(String[] args) throws IOException {
10         Path defaultFile = Lab7Files.getDefaultPath("tasks", "data.txt", "data.txt");
11         Scanner scanner = new Scanner(System.in);
12         String fileName = InputUtils.readLineDefault(scanner, "Enter file path", defaultFile.toString());
13         System.out.println("File size in bytes: " + getFileSize(Path.of(fileName)));
14     }
15 }
```

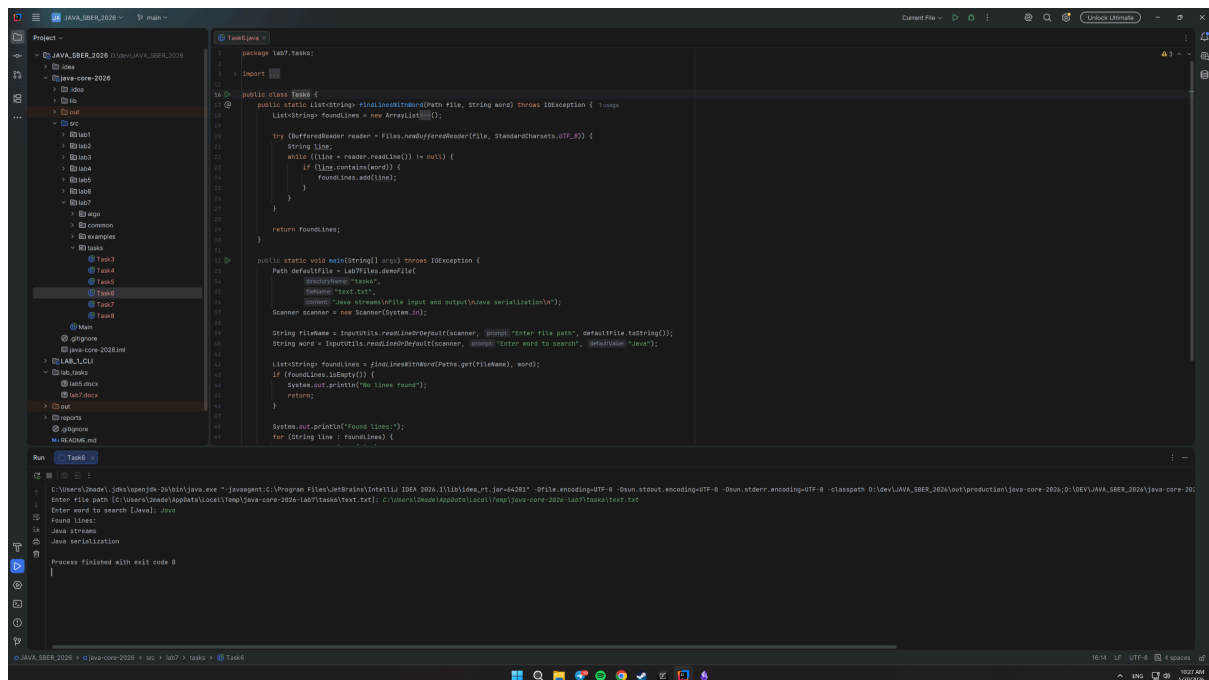
Run Task5

```
C:\Users\Izabela> javac -d bin\src Task5.java
C:\Users\Izabela> java -cp bin\src Task5
Enter file path C:\Users\Izabela\AppData\Local\Temp\java-core-2026-lab7\task5\data.txt
File size in bytes: 19
Process finished with exit code 0
```

3.4 Задание 6. Поиск строк, содержащих слово

Файл с решением: [java-core-2026/src/lab7/tasks/Task6.java](#)

Программа читает файл построчно и добавляет в список все строки, содержащие введенное слово. Если совпадений нет, выводится сообщение **No lines found**.



```
package lab7.tasks;

import java.io.*;
import java.util.*;

public class Task6 {

    public static List<String> findLinesWithWord(Path file, String word) throws IOException {
        List<String> foundLines = new ArrayList<>();

        try (BufferedReader reader = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
            String line;
            while ((line = reader.readLine()) != null) {
                if (line.contains(word)) {
                    foundLines.add(line);
                }
            }
        }

        return foundLines;
    }

    public static void main(String[] args) throws IOException {
        Path defaultFile = Lab7Files.newFile(
            "task6",
            "text.txt",
            "task6",
            "text.txt"
        );
        Scanner scanner = new Scanner(System.in);

        String filename = InputUtil.readLine(scanner, "Enter file path", defaultFile.toString());
        String word = InputUtil.readLine(scanner, "Enter word to search", defaultFile.toString());

        List<String> foundLines = findLinesWithWord(Path.of(filename), word);

        if (foundLines.isEmpty()) {
            System.out.println("No lines found");
            return;
        }

        System.out.println("Found lines:");
        for (String line : foundLines) {
            System.out.println(line);
        }
    }
}
```

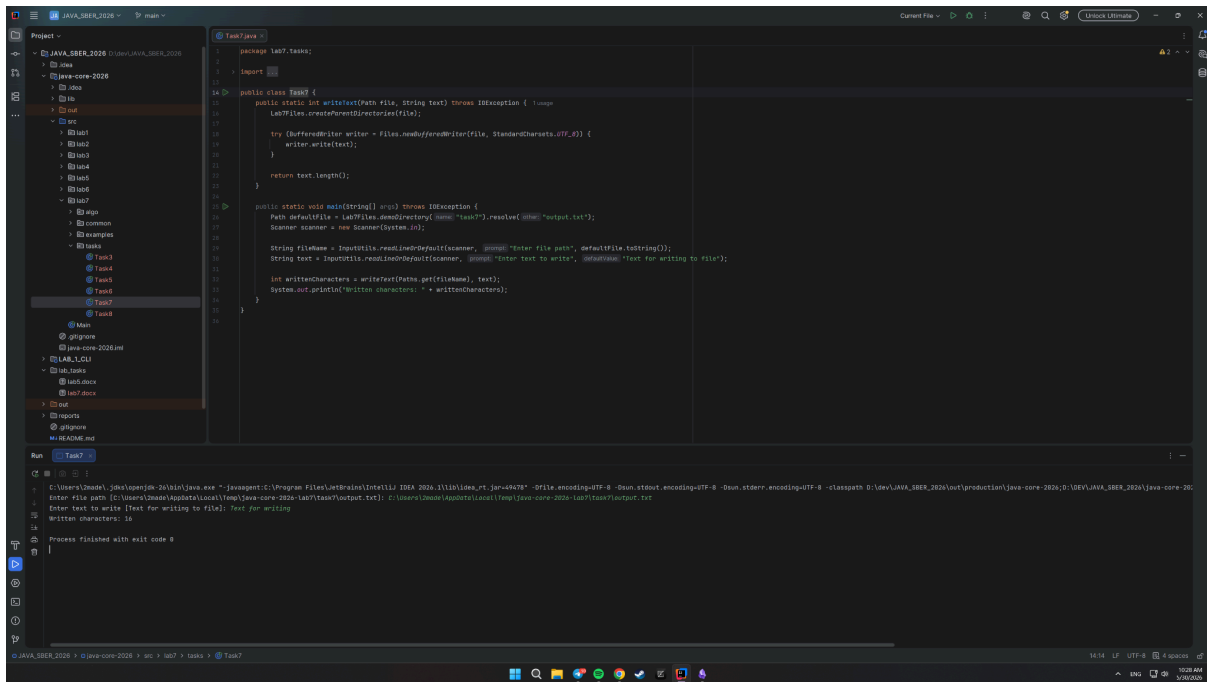
Run console output:

```
C:\Users\masha\Idea\workspace> java -cp .\java-core-2026\src\lab7\tasks\Task6.jar Task6
Enter file path C:\Users\masha\AppData\Local\Temp\java-core-2026-lab7\task6\text.txt
Enter word to search level java
Found lines:
Java streams
Java serialization
Process finished with exit code 0
```

3.5 Задание 7. Запись текста в файл

Файл с решением: [java-core-2026/src/lab7/tasks/Task7.java](#)

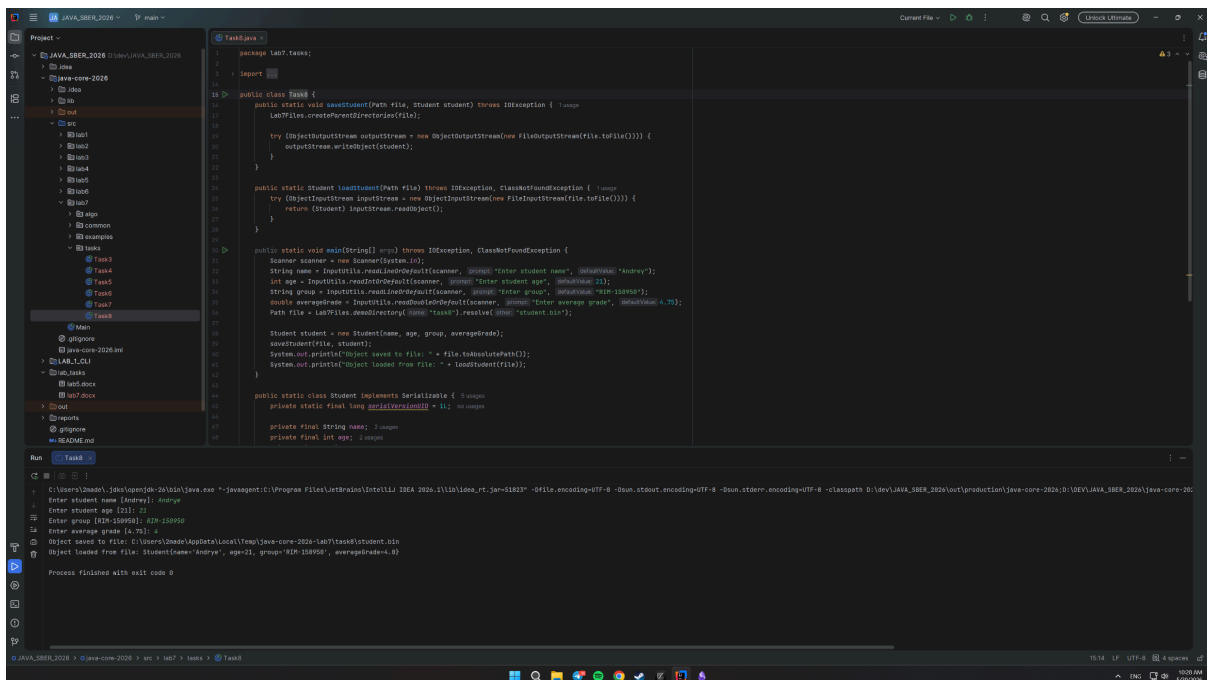
Программа запрашивает путь к файлу и текст для записи. После записи через **BufferedWriter** выводится количество записанных символов.



3.6 Задание 8. Сериализация и восстановление объекта

Файл с решением: [java-core-2026/src/lab7/tasks/Task8.java](#)

Создан класс **Student**, реализующий **Serializable**. Объект сохраняется в файл **student.bin**, затем считывается обратно и выводится на экран.



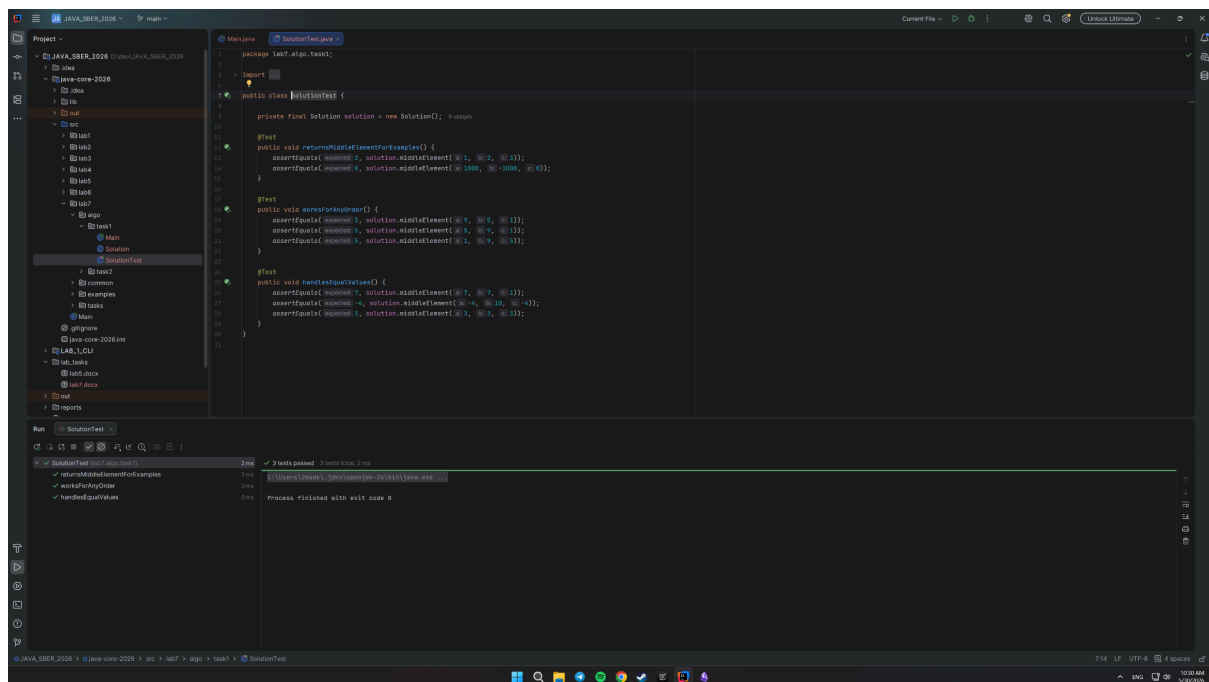
4 Алгоритмические задания

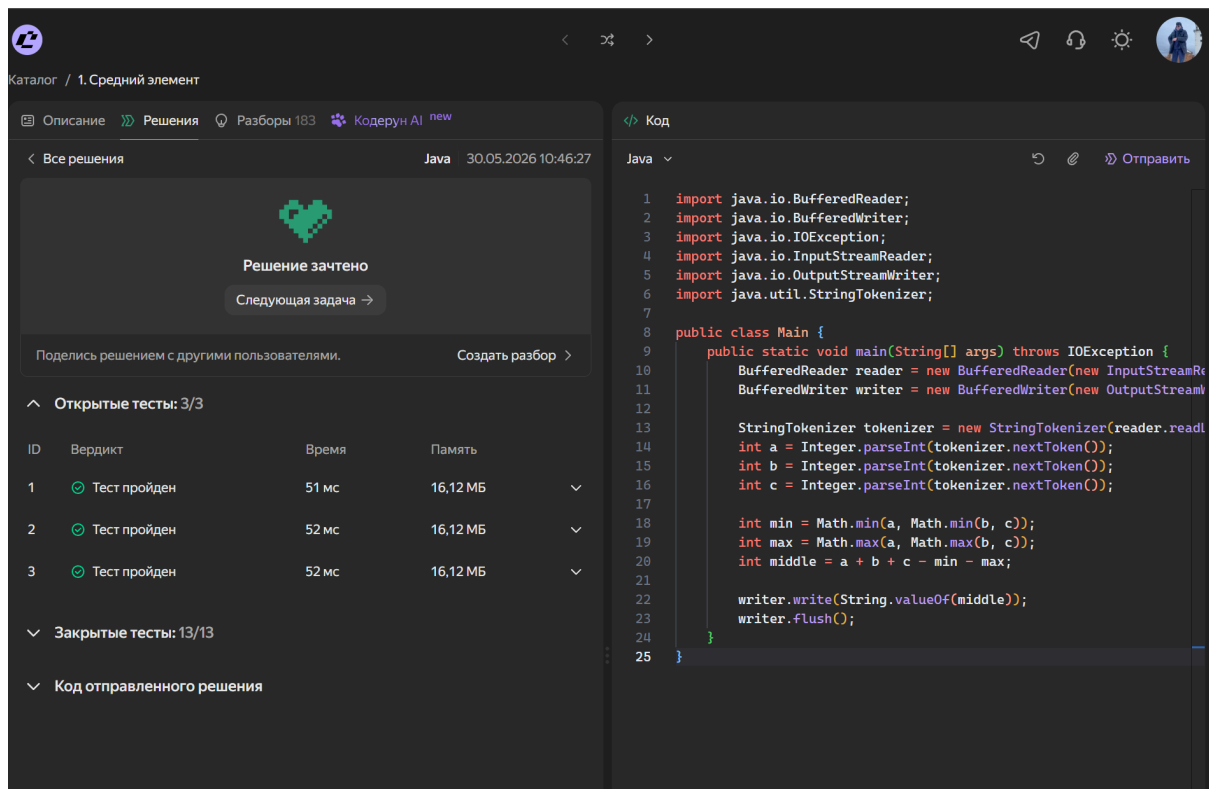
4.1 Алгоритмическая задача 1. «Средний элемент»

Файлы с решением:

- [java-core-2026/src/lab7/algo/task1/Main.java](#)
- [java-core-2026/src/lab7/algo/task1/Solution.java](#)
- [java-core-2026/src/lab7/algo/task1/SolutionTest.java](#)

По условию необходимо считать три целых числа и вывести число, которое после сортировки находится между двумя другими. В решении находится минимум и максимум среди трех чисел, после чего средний элемент вычисляется как $a + b + c - \min - \max$.



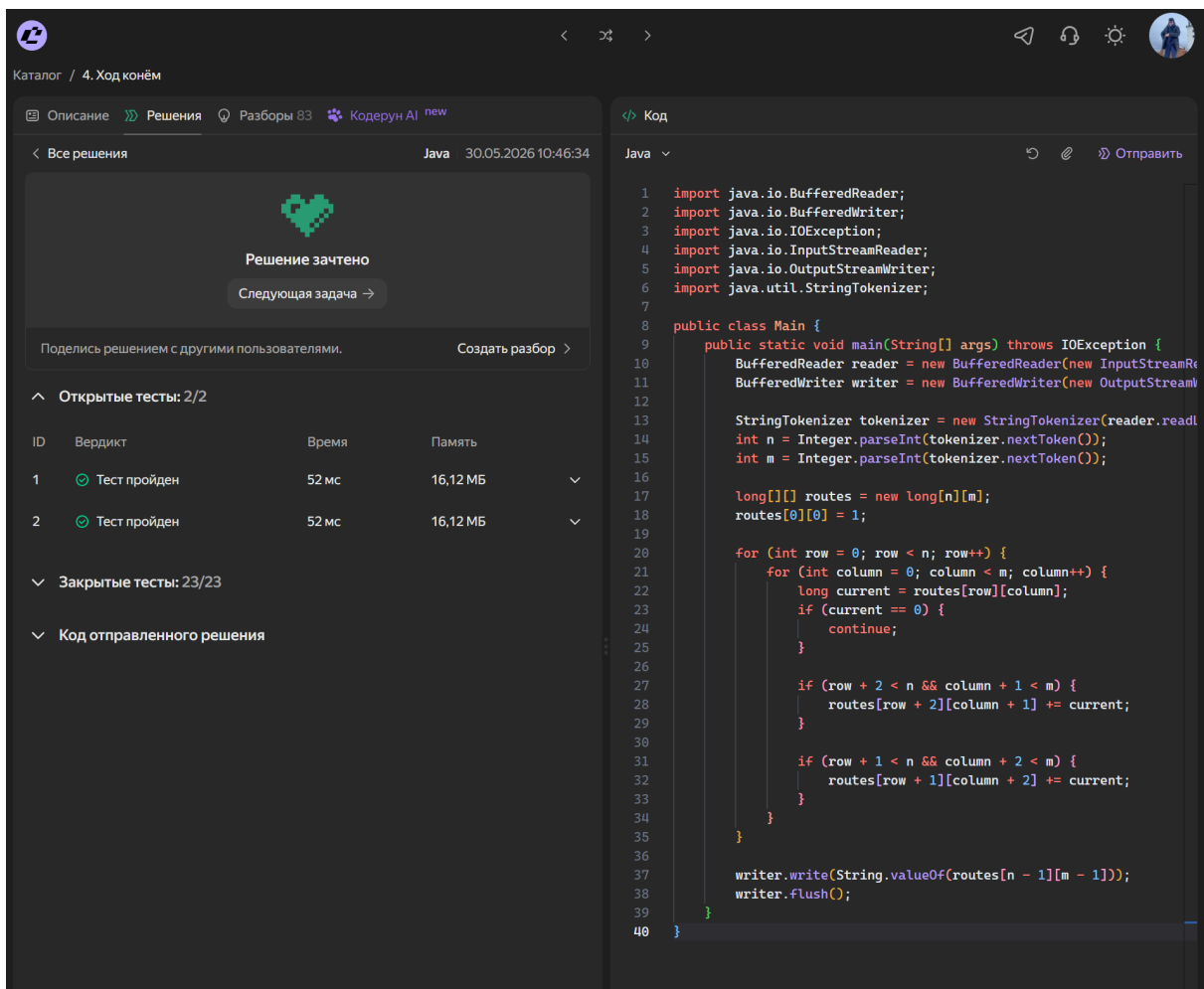
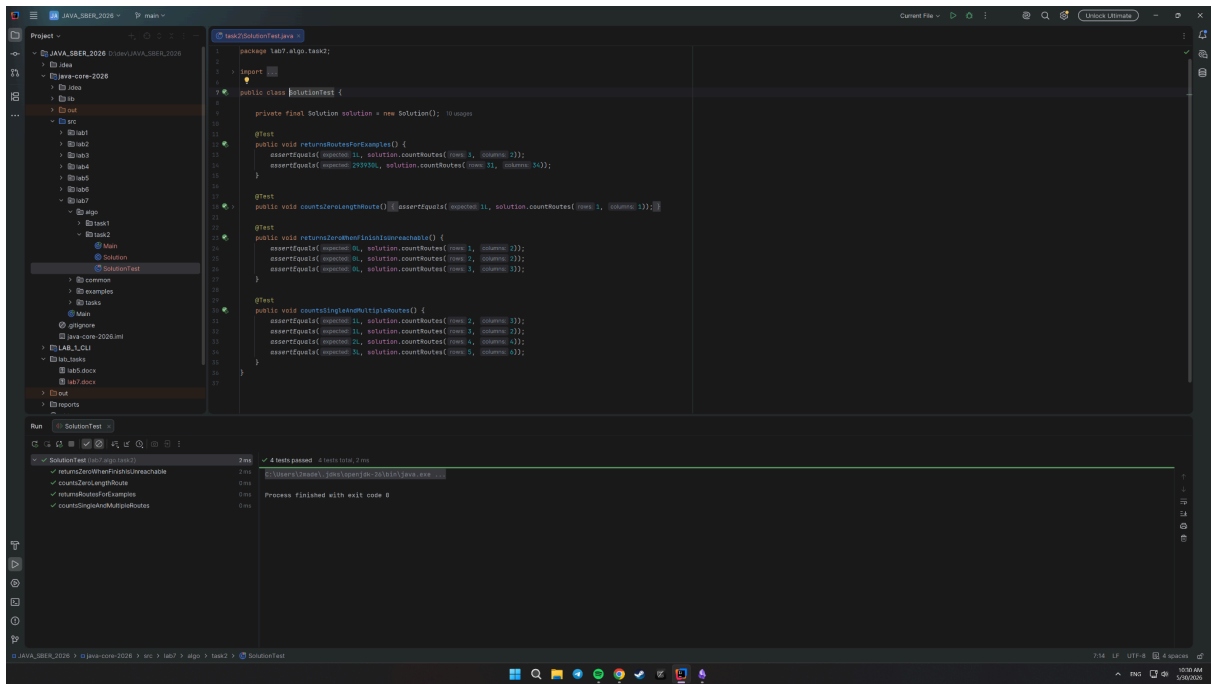


4.2 Алгоритмическая задача 2. «Ход конём»

Файлы с решением:

- [java-core-2026/src/lab7/algo/task2/Main.java](#)
- [java-core-2026/src/lab7/algo/task2/Solution.java](#)
- [java-core-2026/src/lab7/algo/task2/SolutionTest.java](#)

Дана доска $N \times M$. Конь находится в левом верхнем углу и должен попасть в правый нижний угол, двигаясь только на одну клетку вправо и две вниз либо на две клетки вправо и одну вниз. Для подсчета количества маршрутов используется динамическое программирование: в массиве `routes` хранится количество способов попасть в каждую клетку.



Вывод

В ходе лабораторной работы были изучены и реализованы основные способы работы с файлами в Java: создание каталогов и файлов через `File`, байтовый ввод и вывод через `FileInputStream` и `FileOutputStream`, символьный ввод и вывод через `FileReader` и `FileWriter`, буферизированные потоки, преобразование байтовых потоков в символьные с указанием кодировки, запись через `PrintWriter`, а также сериализация и десериализация объектов.

Дополнительно были реализованы самостоятельные задания по подсчету строк, копированию файлов, определению размера файла, поиску строк по слову, записи текста в файл и сохранению объекта класса в бинарный файл. В алгоритмической части решены задачи «Средний элемент» и «Ход конём», для которых подготовлены JUnit-тесты. Все программы были скомпилированы и запущены, тесты завершились успешно.